

A. INVENTAIRE MATÉRIEL

```
[SUDO] dmidecode
```

```
DESCRIPTION : Lit les tables DMI/SMBIOS du firmware pour inventorier le matériel
              (CPU, RAM, carte mère, BIOS, slots, ports) sans redémarrage.
POURQUOI    : Source d'information hardware fiable même sans OS graphique. Utilisé
              pour vérifier les specs réelles, les numéros de série, les versions BIOS.
CONTEXTE    : Audit matériel, support technique, vérification de conformité.
OPTIONS     :
  -t TYPE    Filtre par type (bios, system, baseboard, chassis,
              processor, memory, cache, connector, slot)
  -t N       Filtre par numéro de type DMI (ex: 4=CPU, 17=RAM)
  -s KEYWORD Extrait une valeur unique (bios-version, bios-date,
              system-manufacturer, system-product-name,
              system-serial-number, system-uuid,
              baseboard-manufacturer, baseboard-product-name,
              processor-family, processor-frequency,
              memory-device-size, memory-device-speed)
  -q         Mode silencieux (sans commentaires)
  -u         Mode non-parsé (données brutes)
  --from-dump FICHIER Lit depuis un dump DMI (pour analyse hors ligne)
EXEMPLES    :
$ dmidecode -t bios           # Version et date du BIOS/UEFI
$ dmidecode -t memory         # Slots RAM, taille, vitesse, fabricant
$ dmidecode -t processor      # CPU: modèle, fréquence, cœurs, socket
$ dmidecode -t baseboard      # Carte mère: modèle, révision, numéro série
$ dmidecode -s system-product-name # Modèle de la machine (ex: ThinkPad X1)
$ dmidecode -s system-serial-number # Numéro de série
$ dmidecode -s bios-version   # Version du BIOS
$ dmidecode | grep -i "serial\|part" # Tous les numéros de série/pièces
$ dmidecode -t 17 | grep -E "Size|Speed|Type|Manufacturer|Part" # RAM détaillée
```

```
[USER] lspci
```

```
DESCRIPTION : Liste tous les périphériques connectés sur les bus PCI/PCIe
              (cartes graphiques, réseau, son, stockage, USB, etc.).
POURQUOI    : Identifier le matériel présent, vérifier la détection par le noyau,
              trouver les ID pour les pilotes, diagnostiquer des problèmes matériels.
CONTEXTE    : Installation de pilotes, diagnostic matériel, audit système.
OPTIONS     :
  -v         Affichage verbeux (pilote utilisé, ressources)
  -vv        Très verbeux
  -vvv       Maximum de détails
  -k         Affiche le pilote noyau utilisé et les modules dispo
  -n         Affiche les ID numériques (vendor:device) au lieu des noms
  -nn        Affiche noms ET ID numériques
  -s [domaine:][bus:][device.func] Filtre par emplacement PCI
  -d [vendor]:[device]             Filtre par ID vendor/device
  -t         Vue arborescente (topologie des bus)
  -b         Adresses depuis la vue du bus (pas du CPU)
  -D         Affiche toujours le domaine PCI
  -mm        Format machine-parseable
  -i FICHIER Utilise une base de données PCI IDs alternative
  -Q         Interroge la base centrale pci.ids en ligne
EXEMPLES    :
$ lspci      # Liste rapide de tous les périphériques
$ lspci -v   # Liste avec détails (pilotes, ressources)
$ lspci -k   # Quel pilote gère chaque périphérique
$ lspci -nn  # Avec IDs numériques (utile pour chercher des pilotes)
$ lspci -k | grep -A3 "VGA\|3D" # Carte graphique et son pilote
$ lspci -k | grep -A3 "Network\|Ethernet\|Wireless" # Cartes réseau
$ lspci -t   # Topologie des bus PCI
$ lspci -d 10de: # Tous les périphériques NVIDIA
```

```
[USER] lsusb
```

```
DESCRIPTION : Liste tous les périphériques USB connectés au système
               avec leurs identifiants vendor/product.
POURQUOI    : Identifier des périphériques USB non reconnus, vérifier la détection,
               trouver les ID pour l'écriture de règles udev ou de pilotes.
CONTEXTE    : Diagnostic USB, configuration de périphériques, écriture de règles udev.
OPTIONS     :
    -v                Affichage verbeux (descripteurs complets)
    -s [[bus]:][devnum] Filtre par bus et/ou numéro de périphérique
    -d [vendor]:[product] Filtre par ID vendor/product
    -t                Vue arborescente (topologie USB)
    -D FICHIER        Lit depuis un fichier de périphérique /dev/bus/usb/...
EXEMPLES    :
    $ lsusb           # Liste tous les périphériques USB
    $ lsusb -t        # Topologie arborescente (hubs, vitesses)
    $ lsusb -v        # Détails complets (fabricant, produit, série)
    $ lsusb -d 046d:   # Tous les périphériques Logitech
    $ lsusb | grep -i "storage\|disk\|flash" # Périphériques de stockage USB
    $ watch -n1 lsusb # Surveiller les connexions/déconnexions USB
```

```
[USER] lscpu
```

```
DESCRIPTION : Affiche les informations détaillées sur l'architecture CPU
               (modèle, cœurs, threads, fréquences, caches, flags, NUMA, vulnérabilités).
POURQUOI    : Connaître exactement le CPU sans ouvrir le boîtier. Essentiel pour
               optimiser la compilation, configurer l'affinité, diagnostiquer les
               vulnérabilités Spectre/Meltdown.
CONTEXTE    : Optimisation des performances, sécurité (vulnérabilités CPU), diagnostic.
OPTIONS     :
    -a / --all        Inclut les CPU hors ligne (dans les VM)
    -b / --online     Seulement les CPU en ligne
    -c / --offline    Seulement les CPU hors ligne
    -e / --extended[=LISTE] Format étendu avec colonnes choisies
    -p / --parse[=LISTE] Format parseable (CSV-like)
    -s / --sysroot RÉPERTOIRE Utilise un répertoire sysroot alternatif
    --bytes           Tailles de cache en octets (pas KiB)
    -J / --json       Sortie JSON
    -y / --physical   Affiche les IDs physiques
EXEMPLES    :
    $ lscpu           # Toutes les infos CPU
    $ lscpu | grep -E "CPU(s)|Thread|Core|Socket|Model name|MHz|Cache"
    $ lscpu -J        # Format JSON (pour scripts)
    $ lscpu | grep -i "vulnerabilit" # Statut Spectre/Meltdown/...
    $ lscpu -e=CPU,NODE,SOCKET,CORE,CACHE,ONLINE # Vue étendue NUMA
```

```
[USER] lsmem
```

```
DESCRIPTION : Affiche la mémoire physique disponible avec les plages d'adresses,
               les blocs en ligne/hors ligne et leur état.
POURQUOI    : Voir la mémoire physique (pas juste la RAM disponible), diagnostiquer
               les trous mémoire, vérifier le hot-plug RAM dans les serveurs.
CONTEXTE    : Diagnostic mémoire, NUMA, serveurs avec RAM dynamique.
OPTIONS     :
    -a / --all        Liste tous les blocs mémoire (même hors ligne)
    -o / --output LISTE Colonnes à afficher (RANGE,SIZE,STATE,REMOVABLE,BLOCK,NODE,ZONES)
    -s / --split LISTE Divise par les colonnes spécifiées
    --summary[=MODE]  Résumé uniquement (never, always, only)
    -J / --json       Sortie JSON
    -P / --pairs      Format clé=valeur
    --bytes           Affiche en octets
EXEMPLES    :
    $ lsmem           # Vue condensée de la mémoire physique
    $ lsmem -a        # Tous les blocs mémoire
    $ lsmem -o RANGE,SIZE,STATE,NODE # Colonnes spécifiques
    $ lsmem --summary=only # Juste le résumé (total online/offline)
```

```
[USER] lsblk
```

```
DESCRIPTION : Liste les périphériques de blocs (disques, partitions, LVM, RAID,
               NVMe, boucles) sous forme arborescente avec leurs propriétés.
POURQUOI    : Vue claire de tous les stockages : disques physiques, partitions,
```

```

volumes logiques, points de montage, tailles, types de système de fichiers.
CONTEXTE      : Diagnostic stockage, préparation partitionnement, vérification montages.
OPTIONS       :
-a / --all           Inclut les périphériques vides
-b / --bytes         Taille en octets
-d / --nodeps        Pas les partitions/enfants
-e / --exclude LISTE Exclut les majeurs de périphérique
-f / --fs            Infos système de fichiers (TYPE, LABEL, UUID, MOUNTPPOINT)
-i / --ascii         Caractères ASCII pour l'arbre
-J / --json          Sortie JSON
-l / --list          Format liste (pas arbre)
-m / --perms         Infos permissions (OWNER, GROUP, MODE)
-n / --noheadings    Sans en-têtes
-o / --output LISTE  Colonnes : NAME,MAJ:MIN,RM,SIZE,RO,TYPE,MOUNTPPOINT,
                    FSTYPE,LABEL,UUID,VENDOR,MODEL,SERIAL,STATE,HOTPLUG,
                    TRAN,HCTL,ROTA,DISC-ALN,DISC-GRAN,DISC-MAX,RAND
-p / --paths         Chemins complets (/dev/sdal pas sdal)
-r / --raw           Format brut
-s / --inverse       Inverse la dépendance (depuis partition vers disque)
-S / --scsi          Infos SCSI
-t / --topology      Infos topologie
-x / --sort COLONNE  Trie par colonne
-z / --zoned         Infos des zones
EXEMPLES        :
$ lsblk              # Vue arborescente standard
$ lsblk -f            # Avec systèmes de fichiers, labels, UUID
$ lsblk -o NAME,SIZE,TYPE,FSTYPE,LABEL,MOUNTPPOINT # Colonnes utiles
$ lsblk -J            # JSON pour scripts
$ lsblk -d -o NAME,SIZE,ROTA,TRAN,MODEL,SERIAL # Disques physiques uniquement
$ lsblk /dev/sda      # Un seul disque et ses partitions

```

B. DIAGNOSTIC STOCKAGE & SYSTÈMES DE FICHIERS

```
[SUDO] smartctl
```

```

DESCRIPTION : Interface avec la technologie S.M.A.R.T. des disques durs et SSD pour
surveiller leur santé, lire les attributs et déclencher des auto-tests.
POURQUOI    : Prédire les pannes disque AVANT qu'elles arrivent. Un disque peut avoir
des secteurs défectueux ou des erreurs silencieuses sans symptôme visible.
CONTEXTE     : Maintenance préventive, diagnostic de pannes, validation matériel neuf.
OPTIONS      :
-i DEVICE    Infos générales (modèle, firmware, capacité, SMART support)
-H DEVICE    Statut de santé SMART global (PASSED / FAILED)
-a DEVICE    Rapport complet (infos + attributs + log d'erreurs + tests)
-A DEVICE    Attributs SMART uniquement
-l error DEVICE Journal des erreurs
-l selftest DEVICE Journal des auto-tests
-l xerror DEVICE Journal d'erreurs étendu
-l xselftest DEVICE Journal de tests étendu
-l devstat DEVICE Statistiques du périphérique
-l scttemp DEVICE Historique de température (SCT)
-l scttemphist DEVICE Historique complet de température
-t short DEVICE Lance un auto-test court (~2 min)
-t long DEVICE Lance un auto-test complet (heures)
-t conveyance DEVICE Test de transport (vérifie dommages de livraison)
-t select,N-M DEVICE Test sélectif d'une plage de secteurs
-X DEVICE    Annule un test en cours
-s on DEVICE Active SMART
-s off DEVICE Désactive SMART
-d TYPE      Type de disque (auto, ata, scsi, nvme, sat, 3ware,N...)
--json       Sortie JSON
--quietmode=errorsonly Silencieux sauf erreurs
ATTRIBUTS SMART CRITIQUES :
5   Reallocated_Sector_Ct   Secteurs réalloués (>0 = danger)
187 Reported_Uncorrect      Erreurs UNC non corrigées (>0 = danger)
188 Command_Timeout        Timeouts de commandes
196 Reallocated_Event_Count Événements de réallocation
197 Current_Pending_Sector  Secteurs en attente de réallocation (>0 = à surveiller)
198 Offline_Uncorrectable   Secteurs irrécupérables (>0 = danger)
199 UDMA_CRC_Error_Count    Erreurs câble/interface SATA
EXEMPLES      :
$ smartctl -H /dev/sda      # Santé globale (PASSED ou FAILED)

```

```

$ smartctl -a /dev/sda # Rapport complet
$ smartctl -A /dev/sda | grep -E "^ 5|187|197|198" # Attributs critiques
$ smartctl -t short /dev/sda # Lance test court
$ smartctl -l selftest /dev/sda # Résultats des derniers tests
$ smartctl -i /dev/nvme0 # Infos SSD NVMe
$ smartctl -a /dev/sdb -d sat # Disque USB via adaptateur SAT

```

[SUDO] badblocks

DESCRIPTION : Recherche les secteurs défectueux sur un disque ou une partition en lisant, écrivant et vérifiant les données.

POURQUOI : Détecter les secteurs physiquement défectueux. SMART peut les manquer si le disque ne les a pas encore marqués. Utile sur les vieux disques.

CONTEXTE : Diagnostic matériel, récupération de données, validation avant utilisation.

AVERTISSEMENT: Le mode écriture (-w) DÉTRUIT toutes les données !

OPTIONS :

- v Mode verbeux (affiche la progression)
- s Affiche la progression (pour scripts)
- n Test non-destructif lecture/écriture (lent)
- w Test destructif (écrit et vérifie) – EFFACE LES DONNÉES
- c N Nombre de blocs testés à la fois (défaut: 64)
- b N Taille de bloc en octets
- i FICHIER Lit une liste de blocs à exclure (déjà défectueux)
- o FICHIER Écrit les blocs défectueux dans un fichier
- p N Nombre de passes (défaut: 1)
- t MOTIF Motif de test (pour -w)
- X Mode exclusif (permet test sur système de fichiers monté)

[DÉBUT] [FIN] Plage de blocs à tester

EXEMPLES :

```

$ badblocks -v /dev/sdb # Test lecture seule (non destructif)
$ badblocks -v -o bad_blocks.txt /dev/sdb # Sauvegarde les blocs défectueux
$ badblocks -n /dev/sdb # Test non-destructif lecture/écriture (lent)
$ badblocks -v /dev/sdb 1000000 0 # Teste les 1M premiers blocs seulement
# Intégration avec mkfs (marque les blocs défectueux à la création) :
$ badblocks -v /dev/sdb > bad.txt && mkfs.ext4 -l bad.txt /dev/sdb

```

[USER] testdisk

DESCRIPTION : Outil de récupération de partitions et de réparation de tables de partition corrompues (MBR, GPT, FAT, NTFS, ext2/3/4...).

POURQUOI : Récupérer des partitions accidentellement supprimées ou corrompues, réparer une table de partition, reconstruire un MBR.

CONTEXTE : Récupération de données, réparation post-crash, forensique.

OPTIONS :

- /log Crée un fichier de log testdisk.log
- /debug Mode debug (très verbeux)
- /list Liste les partitions sans interface interactive
- DEVICE Périphérique à analyser

EXEMPLES :

```

$ testdisk # Interface interactive (recommandé)
$ testdisk /dev/sdb # Sur un périphérique spécifique
$ testdisk /log /dev/sdb # Avec journalisation

```

Flux typique :

1. Sélectionner le disque
2. Choisir le type de table (Intel/EFI GPT/Mac/None/Sun)
3. Analyser → Recherche rapide → Recherche complète si besoin
4. Écrire la table de partition récupérée

[USER] photorec

DESCRIPTION : Récupère des fichiers effacés depuis des disques, cartes SD, clés USB, même si le système de fichiers est corrompu. Reconnaît 480+ types de fichiers.

POURQUOI : Récupérer des photos, documents, vidéos supprimés par accident. Fonctionne même quand le système de fichiers est irrécupérable.

CONTEXTE : Récupération de données, forensique numérique.

NOTES : PhotoRec ne restaure PAS les noms de fichiers (numérotation automatique). Nécessite un disque de destination différent de la source.

EXEMPLES :

```

$ photorec /dev/sdb # Récupère depuis /dev/sdb
$ photorec image_disque.img # Depuis une image disque

```

Flux typique :

1. Choisir le disque/partition source

2. Choisir le type de système de fichiers
3. Choisir où sauvegarder (AUTRE DISQUE !)
4. Attendre la récupération

```
[SUDO] ddrescue
```

DESCRIPTION : Clone des disques défectueux en copiant d'abord les zones lisibles et en retentant les zones défectueuses, avec reprise sur interruption.

POURQUOI : Récupérer un maximum de données depuis un disque mourant. Contrairement à dd, ddrescue gère les erreurs intelligemment et peut être interrompu et repris. Outil n°1 pour la récupération de données.

CONTEXTE : Récupération de données, clonage de disques défectueux, forensique.

OPTIONS :

- n Ne pas tenter de réparer les zones défectueuses
- r N Nombre de tentatives sur secteurs défectueux (-1=infini)
- d Lecture directe (bypass cache noyau)
- D Synchronise les données après chaque écriture
- T TEMPS Durée max avant arrêt automatique
- c N Taille du bloc (défaut: 512)
- i OFFSET Offset de départ dans la source
- o OFFSET Offset de départ dans la destination
- s TAILLE Taille à récupérer
- e N Arrêt après N erreurs
- v Verbeux
- vv Très verbeux
- ask Demande confirmation avant d'écrire
- log-events=FICHIER Journal des événements

EXEMPLES :

```
# Clonage en 2 passes (recommandé) :
$ ddrescue -d -r0 /dev/sda sauv.img mapfile.log # Passe 1: zones faciles
$ ddrescue -d -r3 /dev/sda sauv.img mapfile.log # Passe 2: retente les erreurs
# Reprise après interruption (utilise le même mapfile) :
$ ddrescue -d -r1 /dev/sda sauv.img mapfile.log
# Récupération d'une partition :
$ ddrescue -d /dev/sda1 partition.img mapfile.log
# Montage de l'image récupérée :
$ mount -o ro,loop sauv.img /mnt/rescue
```

```
[SUDO] diskscan
```

DESCRIPTION : Scanne un disque pour détecter les erreurs de lecture et mesurer les temps d'accès, identifiant les zones lentes ou défectueuses.

POURQUOI : Détecter un disque en fin de vie par ses temps de lecture anormaux, même avant l'apparition de secteurs SMART défectueux.

CONTEXTE : Maintenance préventive, diagnostic matériel.

EXEMPLES :

```
$ diskscan /dev/sda # Scan complet avec rapport
$ diskscan -v /dev/sda # Verbeux
$ diskscan --no-fix /dev/sda # Scan sans tentative de correction
```

```
[SUDO] ntfsfix
```

DESCRIPTION : Corrige certains problèmes courants sur les systèmes de fichiers NTFS (utilisé sous Linux pour monter des volumes Windows corrompus).

POURQUOI : Un volume NTFS marqué "dirty" (sale) par Windows refuse d'être monté sous Linux. ntfsfix efface ce marqueur pour permettre le montage.

CONTEXTE : Dual-boot Linux/Windows, récupération de données depuis volumes Windows.

AVERTISSEMENT : N'équivaut PAS à chkdsk – ne répare pas les corruptions complexes.

OPTIONS :

- b Efface les mauvais secteurs de la liste
- d Efface le bit "dirty" (cas le plus courant)
- n Simulation (ne modifie rien)

EXEMPLES :

```
$ ntfsfix /dev/sdb1 # Répare le volume NTFS
$ ntfsfix -d /dev/sdb1 # Efface uniquement le bit dirty
$ ntfsfix -n /dev/sdb1 # Test sans modification
```

C. DIAGNOSTIC RÉSEAU

```
[USER] ss
```

```
DESCRIPTION : Affiche l'état des sockets réseau (connexions TCP, UDP, Unix).
               Successeur moderne de netstat, plus rapide et plus complet.
POURQUOI : Diagnostiquer les connexions réseau actives, les ports en écoute,
               trouver quel processus écoute sur un port, détecter les connexions anormales.
CONTEXTE : Diagnostic réseau, sécurité, débogage d'applications réseau.
OPTIONS :
  -t          Sockets TCP
  -u          Sockets UDP
  -x          Sockets Unix (domain sockets)
  -l          Sockets en écoute (LISTEN)
  -a          Toutes les sockets (listen + connectées)
  -n          Numérique (pas de résolution de noms)
  -p          Processus associé (PID, nom)
  -e          Infos étendues (user, inode)
  -i          Infos TCP internes (RTT, fenêtre, retransmissions)
  -s          Statistiques globales
  -m          Infos mémoire des sockets
  -4          IPv4 seulement
  -6          IPv6 seulement
  -r          Résout les noms (inverse de -n)
  -o          Options TCP (keepalive, timers)
  -K          Détruit les sockets correspondant au filtre
dst ADDR:PORT Filtre par destination
src ADDR:PORT Filtre par source
dport OP PORT Filtre par port destination
sport OP PORT Filtre par port source
state ÉTAT Filtre par état (established, listen, time-wait, close-wait...)
EXEMPLES :
$ ss -tlnp          # Ports TCP en écoute avec processus
$ ss -ulnp          # Ports UDP en écoute
$ ss -tnp           # Connexions TCP actives avec processus
$ ss -s             # Statistiques globales
$ ss -tlnp | grep :80 # Qui écoute sur le port 80
$ ss -tnp state established # Connexions établies
$ ss -tnp dst 192.168.1.0/24 # Connexions vers ce sous-réseau
$ ss -xnp           # Sockets Unix (inter-processus)
$ ss -i             # Infos TCP internes (RTT, congest.)
```

```
[USER] tcptraceroute
```

```
DESCRIPTION : Version de traceroute utilisant des paquets TCP SYN au lieu d'ICMP/UDP.
               Passe à travers la plupart des firewalls qui bloquent ICMP.
POURQUOI : traceroute classique est souvent bloqué. tcptraceroute simule une vraie
               connexion TCP et passe là où ICMP est filtré, permettant un diagnostic
               réseau précis dans des environnements sécurisés.
CONTEXTE : Diagnostic réseau en environnement firewallé, débogage de connectivité.
OPTIONS :
  -n          Numérique (pas de résolution DNS)
  -p PORT     Port de destination (défaut: 80)
  -q N        Nombre de sondes par saut (défaut: 3)
  -m TTL      TTL maximum (défaut: 30)
  -w N        Timeout en secondes (défaut: 3)
  -i IFACE    Interface source
  -s ADDR     Adresse source
  -f TTL      TTL de départ
  -t TOS      Type of Service
  -S          N'utilise pas SYN (utilise ACK)
  -E          N'envoie pas ECN
EXEMPLES :
$ tcptraceroute google.com          # Trace vers port 80 (HTTP)
$ tcptraceroute -n google.com 443   # Trace HTTPS (port 443)
$ tcptraceroute -n 192.168.1.1 22   # Trace SSH vers un hôte local
$ tcptraceroute -n -q 1 google.com  # 1 seule sonde par saut (plus rapide)
```

```
[SUDO] tcpdump
```

```
DESCRIPTION : Capture et analyse les paquets réseau en temps réel ou depuis un fichier.
               L'outil de diagnostic réseau de référence en ligne de commande.
POURQUOI : Voir exactement ce qui passe sur le réseau : identifier des problèmes
               de connexion, analyser des protocoles, détecter des intrusions,
               déboguer des applications réseau.
```

CONTEXTE : Diagnostic réseau, sécurité, développement réseau, forensique.

OPTIONS :

-i IFACE Interface (any = toutes)
-w FICHER Écrit dans un fichier pcap
-r FICHER Lit depuis un fichier pcap
-n Pas de résolution DNS
-nn Pas de résolution DNS ni de service
-v / -vv / -vvv Verbose croissante
-c N Capture N paquets puis s'arrête
-s N Taille snapshot (0 = illimité, défaut: 262144)
-A Affiche le contenu en ASCII
-X Affiche en hex et ASCII
-e Affiche les en-têtes Ethernet (adresses MAC)
-l Sortie ligne par ligne (pour pipe vers grep)
-q Mode silencieux (moins de détails)
-t / -tt / -ttt / -tttt Format d'horodatage (relatif, absolu, delta...)
-D Liste les interfaces disponibles
-Z USER Change d'utilisateur après capture
--immediate-mode Réduit la mise en tampon (temps réel)

FILTRES BPF (Berkeley Packet Filter) :

host ADDR Hôte source ou destination
src host ADDR Source uniquement
dst host ADDR Destination uniquement
port N Port source ou destination
src port N Port source
dst port N Port destination
net RÉSEAU/MASQUE Réseau (ex: 192.168.0.0/24)
proto PROTOCOLE Protocole (tcp, udp, icmp, arp, ip6...)
tcp Seulement TCP
udp Seulement UDP
icmp Seulement ICMP
ether host MAC Adresse MAC
and / or / not Combinaison de filtres
greater N Paquets > N octets
less N Paquets < N octets

EXEMPLES :

\$ tcpdump -i eth0 # Tout capturer sur eth0
\$ tcpdump -i any -n port 80 # Trafic HTTP (toutes interfaces)
\$ tcpdump -i eth0 host 192.168.1.100 # Trafic vers/depuis un hôte
\$ tcpdump -i eth0 -w capture.pcap # Sauvegarder pour Wireshark
\$ tcpdump -r capture.pcap # Analyser un fichier existant
\$ tcpdump -i eth0 'tcp[tcpflags] & (tcp-syn) != 0' # Paquets SYN seulement
\$ tcpdump -i eth0 -A port 80 # Voir le contenu HTTP en clair
\$ tcpdump -i eth0 not port 22 # Tout sauf SSH
\$ tcpdump -i eth0 icmp # Pings seulement
\$ tcpdump -c 100 -w out.pcap -i any # 100 premiers paquets, toutes interfaces

```
[USER] idiag-socket-details
```

DESCRIPTION : Affiche des informations détaillées sur les sockets réseau via l'interface de diagnostic du noyau Linux (inet_diag).

POURQUOI : Accéder aux métriques internes TCP (files d'attente, timers, paramètres de congestion) pour un diagnostic approfondi.

CONTEXTE : Débogage réseau avancé, optimisation TCP.

EXEMPLES :

\$ idiag-socket-details # Détails de toutes les sockets actives

D. DIAGNOSTIC PROCESSUS & APPELS SYSTÈME

```
[USER] strace
```

DESCRIPTION : Intercepte et affiche tous les appels système effectués par un processus. Outil fondamental pour comprendre ce que fait un programme.

POURQUOI : Diagnostiquer pourquoi un programme échoue (fichiers introuvables, permissions refusées, ressources manquantes), tracer les accès disque/réseau, comprendre des comportements inattendus sans avoir le code source.

CONTEXTE : Débogage, sécurité, développement, analyse de malware, forensique.

OPTIONS :

-p PID Attache à un processus existant
-e trace=LISTE Filtre les appels système (open, read, write, socket...)
-e trace=file Tous les appels liés aux fichiers

-e trace=network	Tous les appels réseau
-e trace=process	Création/destruction de processus
-e trace=signal	Signaux
-e trace=ipc	IPC (pipes, shm, sémaphores)
-e trace=memory	Gestion mémoire (mmap, brk...)
-e trace=desc	Descripteurs de fichiers
-e trace=stat	Appels stat()
-e fault=APPEL:error=ERREUR	Injecte une erreur dans un appel (fuzzing)
-o FICHIER	Sortie dans un fichier
-f	Suit les processus enfants (fork)
-ff -o PREFIX	Suit les enfants, fichier séparé par PID
-c	Compte les appels (statistiques)
-C	Compte + affiche les appels
-T	Affiche le temps passé dans chaque appel
-t	Timestamp pour chaque appel
-tt	Timestamp microseconde
-ttt	Timestamp Unix
-r	Temps relatif entre appels
-y	Résout les descripteurs en chemins
-yy	Résout aussi les adresses socket
-v	Verbose (structures complètes)
-s N	Taille max des chaînes (défaut: 32)
-x	Hex pour les chaînes non-ASCII
-z	Affiche seulement les appels réussis
-Z	Affiche seulement les appels échoués
COMMANDE [ARGS]	Exécute et trace la commande

EXEMPLES :

```

$ strace ls /tmp                # Tracer l'exécution de ls
$ strace -p 1234                # Attacher à un processus existant
$ strace -e trace=open,read,write ls  # Seulement ouverture/lecture/écriture
$ strace -e trace=file ls 2>&1 | grep "No such" # Fichiers introuvables
$ strace -c ls /tmp             # Statistiques (comptage des appels)
$ strace -f -o trace.log -p 1234 # Trace avec enfants, dans un fichier
$ strace -T -e trace=network curl http://example.com # Temps des appels réseau
$ strace -e trace=openat ls 2>&1 | grep -v "ENOENT" # Fichiers ouverts avec succès

```

```
[USER] strace-log-merge
```

DESCRIPTION : Fusionne plusieurs fichiers de log strace (générés avec -ff -o PREFIX) en un seul log trié par ordre chronologique.

POURQUOI : Quand on trace un processus multi-threads avec strace -ff, on obtient un fichier par thread. strace-log-merge les réunit pour l'analyse.

CONTEXTE : Analyse de traces strace multi-threads.

EXEMPLES :

```

# Générer les logs séparés :
$ strace -ff -ttt -o trace_prefix mon_programme
# Les fusionner :
$ strace-log-merge trace_prefix > trace_complete.log

```

```
[USER] ltrace
```

DESCRIPTION : Intercepte et affiche les appels aux bibliothèques partagées (libc, etc.) effectués par un processus. Complément à strace pour le débogage.

POURQUOI : Voir quelles fonctions de bibliothèque un programme appelle (malloc, printf, fopen...) sans décompiler le binaire.

CONTEXTE : Débogage, analyse de bibliothèques, reverse engineering.

OPTIONS :

-p PID	Attache à un processus existant
-l BIBLIOTHÈQUE	Trace seulement cette bibliothèque
-e FONCTION	Filtre par nom de fonction (wildcards acceptés)
-o FICHIER	Sortie dans un fichier
-f	Suit les processus enfants
-c	Compte (statistiques)
-n N	Indentation pour les appels imbriqués
-i	Affiche l'adresse de l'instruction
-C	Démangle les noms C++
-A N	Taille max des tableaux affichés
-s N	Taille max des chaînes (défaut: 32)

EXEMPLES :

```

$ ltrace ls                    # Trace ls et ses appels de bibliothèque
$ ltrace -p 1234              # Attacher à un processus
$ ltrace -e malloc,free -p 1234 # Seulement les allocations mémoire
$ ltrace -c mon_programme     # Statistiques des appels

```



```
[USER] lsof
```

DESCRIPTION : "List Open Files" – liste tous les fichiers ouverts par les processus (fichiers, sockets, pipes, devices, bibliothèques).

POURQUOI : Diagnostiquer pourquoi un fichier/disque ne peut pas être démonté (processus qui l'utilise encore), trouver les connexions réseau d'un processus, voir les ressources utilisées.

CONTEXTE : Diagnostic système, sécurité, débogage d'applications.

OPTIONS :

-p PID[,PID...]	Filtre par PID
-u USER	Filtre par utilisateur
-c COMMANDE	Filtre par nom de commande (début)
+d RÉPERTOIRE	Fichiers ouverts dans ce répertoire (1 niveau)
+D RÉPERTOIRE	Fichiers ouverts dans ce répertoire (récursif)
-i [TYPE][ADDR][@HOST][:SVC]	Connexions réseau (ex: -i TCP, -i :80, -i @host)
-i 4	IPv4 seulement
-i 6	IPv6 seulement
-i TCP	Seulement TCP
-n	Pas de résolution DNS
-P	Pas de résolution de port (numérique)
-t	Affiche seulement les PIDs (pour scripts)
-F CHAMPS	Format pour parsing (p=pid, c=cmd, n=name, t=type...)
-r [N]	Répète toutes les N secondes
-w	Supprime les messages d'avertissement
+Ll	Fichiers avec 0 liens (déjà supprimés mais ouverts)

EXEMPLES :

\$ lsof /chemin/vers/fichier	# Qui a ce fichier ouvert
\$ lsof +D /mnt/disque	# Pourquoi je ne peux pas démonter
\$ lsof -p 1234	# Fichiers ouverts par le PID 1234
\$ lsof -u alice	# Fichiers ouverts par l'utilisateur alice
\$ lsof -i :80	# Qui écoute/utilise le port 80
\$ lsof -i TCP -n -P	# Toutes les connexions TCP
\$ lsof -i UDP	# Toutes les connexions UDP
\$ lsof -c nginx	# Fichiers ouverts par nginx
\$ lsof +Ll	# Fichiers supprimés mais encore ouverts (récupération espace)
\$ lsof -t -i :80 xargs kill	# Tuer tous les processus sur port 80

```
[USER] dbus-monitor
```

DESCRIPTION : Surveille en temps réel les messages D-Bus transitant sur le bus système ou de session, pour diagnostiquer les communications inter-processus.

POURQUOI : Comprendre les événements D-Bus (notifications, changements d'état, appels de méthodes) entre les composants du système (NetworkManager, systemd, GNOME, applications...).

CONTEXTE : Débogage d'applications, développement D-Bus, diagnostic système.

OPTIONS :

--system	Bus système (/run/dbus/system_bus_socket)
--session	Bus de session (défaut, par utilisateur)
--address ADDR	Adresse D-Bus spécifique
--profile	Format profil (timestamp + stats)
--monitor	Mode moniteur (reçoit tout sans s'enregistrer)

FILTRE : Filtre D-Bus de correspondance (ex: type='signal')

EXEMPLES :

\$ dbus-monitor	# Surveille le bus de session
\$ dbus-monitor --system	# Bus système (NetworkManager, UPower, etc.)
\$ dbus-monitor --system "type='signal'"	# Seulement les signaux système
\$ dbus-monitor "sender='org.freedesktop.NetworkManager'"	# NM seulement
\$ dbus-monitor "interface='org.freedesktop.DBus.Properties'"	# Changements propriétés
\$ dbus-monitor --system "path='/org/freedesktop/login1'"	# Événements login

E. DIAGNOSTIC BINAIRES & BIBLIOTHÈQUES

```
[USER] ldd
```

DESCRIPTION : Affiche les bibliothèques partagées (.so) dont dépend un exécutable et résout leurs chemins réels sur le système.

POURQUOI : Diagnostiquer les erreurs "library not found", vérifier les dépendances d'un binaire, comprendre pourquoi un programme ne démarre pas.

CONTEXTE : Développement, packaging, diagnostic de dépendances, portage.

AVERTISSEMENT: N'exécuter ldd que sur des binaires de confiance (ldd peut exécuter du code).

```
OPTIONS      :
-v           Verbeux (version des bibliothèques)
-u           Affiche les dépendances inutilisées
-d           Résolution des objets de données
-r           Résolution complète (fonctions + données)
EXEMPLES     :
$ ldd /usr/bin/bash           # Dépendances de bash
$ ldd /usr/bin/python3        # Dépendances de Python
$ ldd mon_programme | grep "not found" # Bibliothèques manquantes
$ ldd -v /usr/bin/gcc          # Avec versions des bibliothèques
# Alternative sûre (sans exécution) :
$ objdump -p mon_programme | grep NEEDED
```

```
[USER] nm
```

```
DESCRIPTION  : Liste les symboles (fonctions, variables) d'un fichier objet,
                bibliothèque statique ou exécutable.
POURQUOI     : Déboguer des erreurs de linkage ("undefined symbol", "multiple definition"),
                vérifier qu'une fonction est bien exportée, analyser la structure d'un binaire.
CONTEXTE     : Développement C/C++, débogage de bibliothèques, reverse engineering.
OPTIONS      :
-a           Inclut les symboles de débogage
-A / --print-file-name  Préfixe chaque symbole du nom de fichier
-C / --demangle          Démangle les noms C++ (lisibles)
-D / --dynamic           Symboles dynamiques (bibliothèques partagées)
-g / --extern-only       Seulement les symboles globaux (exportés)
-n / --numeric-sort      Tri par adresse numérique
-r / --reverse-sort      Tri inverse
-s / --print-armap       Affiche l'index des archives (.a)
-u / --undefined-only     Seulement les symboles non définis (importés)
--defined-only          Seulement les symboles définis
-f FORMAT              Format de sortie (bsd, sysv, posix)
--size-sort             Tri par taille
TYPES DE SYMBOLES :
T (t)  Text (code) – défini dans .text
D (d)  Data – défini dans .data
B (b)  BSS – variable non initialisée
U       Undefined – importé d'une autre bibliothèque
R (r)  Read-only data (.rodata)
W (w)  Weak symbol
V (v)  Weak object
Majuscule = global, minuscule = local
EXEMPLES   :
$ nm /usr/lib/libm.so           # Symboles de libm
$ nm -D /usr/lib/libpthread.so.0 # Symboles dynamiques de libpthread
$ nm -C mon_prog | grep "U "    # Symboles non définis (dépendances)
$ nm -C libmachin.a | grep " T " # Fonctions exportées
$ nm -u mon_programme           # Ce que le programme importe
```

```
[USER] objdump
```

```
DESCRIPTION  : Affiche des informations détaillées sur les fichiers objets ELF :
                désassemblage, en-têtes, sections, symboles, relocations.
POURQUOI     : Analyser des binaires compilés, déboguer à bas niveau, vérifier
                l'architecture cible, désassembler du code, reverse engineering.
CONTEXTE     : Développement bas niveau, sécurité, reverse engineering, débogage.
OPTIONS      :
-d / --disassemble      Désassemble les sections exécutables
-D / --disassemble-all  Désassemble toutes les sections
-S / --source           Mélange code source et assembleur (si debug info)
-f / --file-headers      En-têtes des fichiers
-h / --section-headers   En-têtes des sections
-x / --all-headers       Tous les en-têtes
-p / --private-headers   Informations privées (NEEDED pour .so)
-r / --reloc             Entrées de relocation
-R / --dynamic-reloc     Relocations dynamiques
-s / --full-contents     Contenu complet de toutes les sections
-t / --syms              Table des symboles
-T / --dynamic-syms      Table des symboles dynamiques
-g / --debugging         Informations de débogage
-C / --demangle          Démangle les noms C++
-M OPTION               Options du désassembleur (intel, att, no-aliases...)
```

```

--no-show-raw-insn      Cache les octets bruts des instructions
-j SECTION              Traite seulement cette section
--start-address=ADDR    Commence le désassemblage à cette adresse
--stop-address=ADDR     Arrête à cette adresse
-l / --line-numbers     Affiche les numéros de lignes source
-z / --disassemble-zeroes N'optimise pas les séquences de zéros

```

EXEMPLES :

```

$ objdump -d /usr/bin/ls          # Désassemble ls
$ objdump -D -M intel fichier.o   # Désassemble en syntaxe Intel
$ objdump -h mon_prog             # Sections du binaire
$ objdump -p /usr/lib/libssl.so | grep NEEDED # Dépendances .so (sûr, sans exec)
$ objdump -t mon_prog             # Table des symboles
$ objdump -S -C mon_prog          # Code C intercalé avec assembleur

```

```
[USER] readelf
```

DESCRIPTION : Affiche des informations sur les fichiers au format ELF (Executable and Linkable Format) – l'essentiel des informations de structure du binaire.

POURQUOI : Analyser la structure d'un exécutable ou d'une bibliothèque ELF sans les abstractions de binutils. Plus bas niveau qu'objdump.

CONTEXTE : Développement bas niveau, sécurité, analyse de binaires.

OPTIONS :

```

-a / --all              Toutes les infos
-h / --file-header      En-tête ELF (architecture, type, point d'entrée...)
-l / --program-headers  Headers de programme (segments)
-S / --section-headers  Headers de sections
-s / --syms             Table des symboles
--dyn-syms              Symboles dynamiques
-n / --notes            Section notes (build-id, ABI...)
-r / --relocs           Relocations
-d / --dynamic          Section dynamique (NEEDED, SONAME, RPATH...)
-V / --version-info     Infos de version
-A / --arch-specific    Infos spécifiques à l'architecture
-u / --unwind           Infos de déroulement de pile
-g / --section-groups   Groupes de sections
-t / --section-details  Détails des sections
-e / --headers          Tous les headers (ELF + programme + sections)
-x SECTION             Hex dump d'une section
-p SECTION             String dump d'une section
-R SECTION             Hex dump déchiffré (si chiffré)
-z                     Décompresse les sections compressées
-W / --wide            Format large (évite le retour à la ligne)
-c / --archive-index    Index d'archive (.a)

```

EXEMPLES :

```

$ readelf -h /usr/bin/ls          # En-tête ELF (architecture, type)
$ readelf -d /usr/lib/libssl.so   # Section dynamique (dépendances, SONAME)
$ readelf -s mon_prog            # Symboles
$ readelf -S mon_prog            # Sections et leurs propriétés
$ readelf -n mon_prog            # Build-ID
$ readelf -l mon_prog            # Segments (layout mémoire)

```

```
[USER] addr2line
```

DESCRIPTION : Convertit des adresses mémoire (depuis un crash/core dump) en noms de fichiers source et numéros de ligne, grâce aux infos de débogage.

POURQUOI : Quand un programme crashe avec une adresse mémoire (ex: dans dmesg ou valgrind), addr2line retrouve la ligne de code correspondante.

CONTEXTE : Débogage post-crash, analyse de core dumps, développement.

OPTIONS :

```

-e FICHIER              Fichier exécutable (requis)
-f / --functions        Affiche le nom de la fonction
-C / --demangle         Démangle les noms C++
-s / --basename         N'affiche que le nom du fichier (pas le chemin)
-i / --inlines          Affiche les fonctions inlineées
-a / --addresses        Affiche l'adresse en entrée
-p / --pretty-print     Format lisible sur une ligne
ADRESSE [...]          Adresse(s) à convertir (hex, ex: 0x4005e8)

```

EXEMPLES :

```

$ addr2line -e mon_programme 0x4005e8 # Adresse → fichier:ligne
$ addr2line -e mon_programme -f -C 0x4005e8 # Avec nom de fonction C++
$ addr2line -e mon_programme -f -i 0x4005e8 # Avec fonctions inlineées
# Depuis un crash SIGSEGV dans dmesg :
$ addr2line -e mon_programme -f -C 0x0000000000401234

```

```
[USER] eu-addr2line
```

DESCRIPTION : Version elfutils de addr2line, avec support de debuginfod (téléchargement automatique des symboles de débogage).

POURQUOI : Comme addr2line mais fonctionne avec les symboles de débogage hébergés sur un serveur debuginfod (ex: pour les paquets système).

CONTEXTE : Débogage de binaires système sans installer les paquets -debuginfo.

OPTIONS :

- e FICHIER Exécutable
- f / --functions Nom de la fonction
- s / --base-names Nom de fichier court
- i / --inlines Fonctions inlineées
- debuginfod=URL URL du serveur debuginfod
- p / --pretty-print Format compact
- C / --demangle Démangle C++

EXEMPLES :

```
$ eu-addr2line -e /usr/bin/ls -f 0x1234
$ DEBUGINFOD_URLS="https://debuginfod.fedoraproject.org/" eu-addr2line -e /usr/bin/bash -f 0x5678
```

```
[USER] eu-stack
```

DESCRIPTION : Affiche la pile d'appels (backtrace/stack trace) d'un processus en cours d'exécution ou depuis un core dump (elfutils).

POURQUOI : Voir ce qu'un processus est en train de faire sans GDB, diagnostiquer des blocages ou des boucles infinies.

CONTEXTE : Débogage, analyse de performance, diagnostic de blocages.

OPTIONS :

- p PID PID du processus à examiner
- c CORE Fichier core dump
- e EXÉCUTABLE Exécutable associé
- n N Limite le nombre de frames
- l Threads seulement (pas les processus)
- a Tous les threads
- v Verbeux
- d Utilise debuginfod
- maxframes=N Nombre max de frames par thread

EXEMPLES :

```
$ eu-stack -p 1234 # Backtrace du PID 1234
$ eu-stack -p 1234 -a # Tous les threads
$ eu-stack -c core.1234 -e mon_prog # Depuis un core dump
```

```
[USER] gcore
```

DESCRIPTION : Génère un core dump d'un processus en cours d'exécution SANS le tuer. Capture l'état complet de la mémoire pour analyse ultérieure avec gdb.

POURQUOI : Inspecter l'état interne d'un processus (variables, piles, mémoire) à un instant T, sans interrompre son exécution.

CONTEXTE : Débogage post-mortem, analyse de mémoire, capture d'état.

OPTIONS :

- o PRÉFIXE Préfixe du fichier de sortie (défaut: core)
- PID [PID...] PID(s) à capturer

EXEMPLES :

```
$ gcore 1234 # Génère core.1234
$ gcore -o /tmp/dump 1234 # Génère /tmp/dump.1234
# Analyser le core dump :
$ gdb mon_programme core.1234
(gdb) bt # Backtrace
(gdb) info threads # État de tous les threads
```

F. DIAGNOSTIC SYSTÈME (FIRMWARE, ACPI, MCE)

```
[SUDO] acpidump
```

DESCRIPTION : Extrait les tables ACPI (Advanced Configuration and Power Interface) du firmware UEFI/BIOS vers des fichiers binaires ou texte.

POURQUOI : Analyser les tables ACPI pour diagnostiquer des problèmes d'énergie, de réveil, de gestion thermique, ou de périphériques non détectés.

CONTEXTE : Diagnostic firmware, développement noyau, rapport de bugs matériels.

```

OPTIONS      :
  -o FICHIER      Fichier de sortie (binaire)
  -f FICHIER      Lit depuis un fichier de dump existant
  -n NOM          Extrait une table spécifique (DSDT, SSDT, FADT...)
  -b             Format binaire (fichiers séparés par table)
  -v             Verbeux

EXEMPLES      :
$ acpidump > acpi_tables.out      # Dump de toutes les tables ACPI
$ acpidump -b                    # Fichiers binaires séparés (DSDT.dat, etc.)
$ acpidump -n DSDT -o dsdt.dat    # Table DSDT seulement
# Décompiler avec iasl :
$ acpidump -b && iasl -d dsdt.dat  # Décompile en .dsl lisible

```

```
[USER] acpixtract
```

```

DESCRIPTION   : Extrait des tables ACPI individuelles depuis un dump acpidump
                en fichiers binaires séparés.
POURQUOI      : Préparer les tables ACPI pour la décompilation avec iasl.
CONTEXTE      : Analyse ACPI, développement firmware.
OPTIONS       :
  -a           Extrait toutes les tables
  -l           Liste les tables disponibles (sans extraire)
  -s NOM       Extrait une table spécifique
FICHIER        Fichier dump acpidump

EXEMPLES      :
$ acpixtract -l acpi.out          # Liste les tables présentes
$ acpixtract -a acpi.out          # Extrait tout
$ acpixtract -s DSDT acpi.out     # Extrait seulement DSDT

```

```
[USER] iasl
```

```

DESCRIPTION   : Compilateur/décompilateur ACPI ASL (ACPI Source Language).
                Convertit les tables ACPI binaires en code source lisible et inversement.
POURQUOI      : Lire et comprendre les tables ACPI (DSDT, SSDT), corriger des bugs
                de firmware, ou créer des tables ACPI personnalisées.
CONTEXTE      : Développement firmware, débogage ACPI, correction de bugs matériels.
OPTIONS       :
  -d FICHIER.dat      Décompile une table binaire en .dsl
  -da FICHIER1 FICHIER2... Décompile plusieurs tables en tenant compte des relations
  -dp FICHIER.dat      Décompile et affiche sur stdout
  -e FICHIER.dat      Table externe (pour -da)
FICHIER.asl / FICHIER.dsl Compile en .aml
  -tc                Génère aussi un fichier C (.hex)
  -ta                Génère un fichier assembleur
  -p PRÉFIXE         Préfixe des fichiers de sortie
  -v                Verbeux
  -w N               Niveau d'avertissement (1-4)
  -x N               Niveau des exceptions

EXEMPLES      :
$ iasl -d dsdt.dat      # Décompile DSDT.dat en dsdt.dsl
$ iasl dsdt.dsl         # Compile dsdt.dsl en dsdt.aml
$ cat dsdt.dsl | grep -i "thermal\|fan\|temp" # Cherche la gestion thermique

```

```
[SUDO] mcelog
```

```

DESCRIPTION   : Décode et journalise les erreurs Machine Check Exception (MCE) du CPU,
                qui signalent des défaillances matérielles (mémoire, CPU, bus).
POURQUOI      : Les MCE sont des alertes hardware critiques souvent silencieuses.
                mcelog les décode en messages compréhensibles pour identifier la source
                d'instabilité (RAM défaillante, CPU surchauffé, bus défectueux).
CONTEXTE      : Diagnostic de plantages mystérieux, maintenance serveur, hardware défaillant.
OPTIONS       :
  --daemon        Mode démon (surveillance en continu)
  --client        Communique avec le démon en cours d'exécution
  --ascii         Décode depuis stdin (format texte)
  --file FICHIER  Lit depuis un fichier MCE
  --syslog        Journalise via syslog
  --syslog-error  Journalise les erreurs via syslog
  --no-syslog     Ne pas utiliser syslog
  --logfile FICHIER Fichier de log
  --filter        Filtre les MCE connues comme non-critiques
  --dmi           Inclut les infos DMI dans le rapport

```

```

--cpu TYPE          Type de CPU (pour le décodage)
--cpuinfo           Affiche les infos CPU
--ignoreudev        Ignore les erreurs de périphérique
--unknown-fatal     MCE inconnue = fatale
FICHIER             Décode un fichier MCE binaire (/dev/mcelog)
EXEMPLES            :
$ mcelog             # Décode le log MCE actuel
$ mcelog /dev/mcelog # Depuis le périphérique directement
$ mcelog --client    # Consulte le démon mcelog
$ mcelog --ascii < /var/log/mcelog # Décode un log existant
# Vérification dans les logs :
$ journalctl -k | grep -i "mce\|machine check\|hardware error"
$ dmesg | grep -i "mce\|hardware error"

```

[SUDO] turbostat

DESCRIPTION : Affiche les fréquences CPU, les états C/P, la consommation d'énergie et la température par cœur en temps réel sur les CPU Intel/AMD.

POURQUOI : Diagnostiquer les problèmes de performance CPU (throttling thermique, mauvais P-states, consommation anormale), valider l'overclocking.

CONTEXTE : Optimisation performance, diagnostic thermique, économies d'énergie.

OPTIONS :

- i N / --interval N Intervalle de mesure en secondes (défaut: 5)
- n N / --num_iterations N Nombre d'itérations
- s LISTE / --show LISTE Colonnes à afficher
- S / --Summary Résumé uniquement (pas par cœur)
- c N / --cpu N CPU spécifique
- J / --Joules Énergie en joules
- l / --list Liste les colonnes disponibles
- o FICHIER / --out FICHIER Fichier de sortie
- q / --quiet Minimal
- Séparateur avant la commande à exécuter

COMMANDE : Lance une commande et mesure pendant son exécution

EXEMPLES :

```

$ turbostat             # Mesure continue (Ctrl+C pour arrêter)
$ turbostat -i 1       # Mesure chaque seconde
$ turbostat sleep 10   # Mesure pendant 10 secondes
$ turbostat --show CPU,Avg_MHz,Busy%,Bzy_MHz,TSC_MHz,CoreTmp,PkgWatt
$ turbostat -S sleep 5 # Résumé global seulement

```

[SUDO] cpupower

DESCRIPTION : Configure et diagnostique la gestion d'énergie du CPU (fréquence, gouverneur, états C/P, latence).

POURQUOI : Diagnostiquer les throttlings, configurer le gouverneur de fréquence (performance vs économie d'énergie), voir les capacités réelles du CPU.

CONTEXTE : Optimisation performance, diagnostic énergétique, serveurs et laptops.

SOUS-COMMANDES :

- frequency-info Infos sur la fréquence actuelle et les limites
- frequency-set Configure la fréquence (-f MIN, -u MAX, -g GOUVERNEUR)
- idle-info Infos sur les états C (idle states)
- idle-set Active/désactive des états C
- info Infos générales (boost, gouverneur actuel)
- monitor Surveillance en temps réel des fréquences
- set Configure diverses options

EXEMPLES :

```

$ cpupower frequency-info # Fréquence et gouverneur actuels
$ cpupower frequency-info -c 0 # CPU 0 seulement
$ cpupower idle-info # États C disponibles
$ cpupower monitor # Surveillance temps réel
$ cpupower frequency-set -g performance # Gouverneur "performance"
$ cpupower frequency-set -g powersave # Gouverneur économie d'énergie
$ cpupower frequency-set -d 2GHz -u 4GHz # Limites min/max fréquence
$ cpupower info # Boost actif/inactif

```

[USER] flashrom

DESCRIPTION : Lit, écrit, vérifie et efface les puces de mémoire flash (BIOS/UEFI, EC, GPU VBIOS, NIC firmware) sur diverses interfaces.

POURQUOI : Mettre à jour manuellement un BIOS corrompu, sauvegarder le BIOS original avant modification, récupérer un système non-démarrable.

CONTEXTE : Récupération firmware, coreboot, libreboot, maintenance matériel.

AVERTISSEMENT: Une mauvaise utilisation peut rendre le système non-démarrable !
Toujours sauvegarder avant d'écrire.

OPTIONS :

- p PROGRAMMATEUR Programmeur (internal, linux_spi, ch341a_spi, ...)
- r FICHIER Lit la puce vers un fichier
- w FICHIER Écrit un fichier vers la puce
- v FICHIER Vérifie (compare puce avec fichier)
- E Efface la puce
- c PUCE Force une puce spécifique (ignorer autodétection)
- l FICHIER Fichier de layout (écrire seulement certaines régions)
- i RÉGION Région à opérer (combiné avec -l)
- n Ne vérifie pas après écriture
- noverify-all Ne vérifie pas les blocs non écrits
- ifd Utilise le Intel Firmware Descriptor

EXEMPLES :

```
$ flashrom -p internal                      # Détecte la puce BIOS interne
$ flashrom -p internal -r bios_backup.bin # Sauvegarde du BIOS actuel
$ flashrom -p internal -v nouveau_bios.bin # Vérifie sans écrire
$ flashrom -p internal -w nouveau_bios.bin # Écrit le nouveau BIOS (DANGER!)
$ flashrom -p ch341a_spi -r dump.bin      # Lit via programmeur externe CH341A
```

G. MISE À JOUR FIRMWARE (fwupd)

```
[USER/SUDO] fwupdmgr
```

DESCRIPTION : Gestionnaire de mises à jour firmware via le framework fwupd.
Met à jour le BIOS/UEFI, les SSD NVMe, les hubs USB, les docks Thunderbolt et autres périphériques compatibles LVFS.

POURQUOI : Mettre à jour le firmware de manière sécurisée et standardisée, sans outil propriétaire ni Windows. Corrige des bugs, vulnérabilités de sécurité et améliore la compatibilité.

CONTEXTE : Maintenance système, mise à jour de sécurité (ex: patches Spectre/Intel ME).

COMMANDES :

- get-devices Liste les périphériques gérables par fwupd
- get-updates Vérifie les mises à jour disponibles
- update Installe toutes les mises à jour disponibles
- upgrade Pareil que update
- install FICHIER/URL Installe un firmware spécifique
- downgrade Revient à une version antérieure
- refresh Rafraîchit les métadonnées LVFS
- get-history Historique des mises à jour
- get-releases DEVICE Toutes les versions disponibles pour un périphérique
- get-results DEVICE Résultats de la dernière mise à jour
- activate DEVICE Active un firmware installé (nécessite reboot)
- clear-results DEVICE Efface les résultats
- enable-remote NOM Active un dépôt LVFS
- disable-remote NOM Désactive un dépôt
- modify-remote NOM CLÉ VALEUR Modifie la config d'un dépôt
- get-remotes Liste les dépôts configurés
- security Rapport de sécurité firmware (HSI)
- offline Planifie la mise à jour au prochain démarrage
- assume-yes / -y Pas de confirmation
- no-reboot-check N'insiste pas pour le redémarrage
- verbose Verbeux

EXEMPLES :

```
$ fwupdmgr get-devices                      # Périphériques gérables
$ fwupdmgr refresh                          # Mise à jour des métadonnées
$ fwupdmgr get-updates                      # Voir les mises à jour disponibles
$ fwupdmgr update                          # Mettre à jour tout
$ fwupdmgr update --offline                # Planifier (appliqué au redémarrage)
$ fwupdmgr security                        # Évaluation sécurité firmware (HSI score)
$ fwupdmgr get-history                      # Ce qui a été mis à jour
$ fwupdmgr get-releases "Intel NUC"        # Versions disponibles pour un périphérique
```

H. DÉBOGAGE APPLICATIF AVANCÉ (GDB, COREDUMP)

```
[USER] gdb
```

DESCRIPTION : GNU Debugger – le débogueur de référence pour les programmes C, C++,

Fortran, et autres langages compilés. Permet l'exécution pas à pas, les points d'arrêt, l'inspection de mémoire et l'analyse de core dumps.

POURQUOI : Trouver et corriger des bugs difficiles (SIGSEGV, corruption mémoire, conditions de course), analyser des crashes post-mortem, comprendre le comportement d'un programme à bas niveau.

CONTEXTE : Développement logiciel, analyse de crashes, sécurité, reverse engineering.

OPTIONS (ligne de commande) :

PROGRAMME [CORE]	Lance ou analyse un core dump
-p PID	Attache à un processus existant
--args PROGRAMME ARGS	Lance avec des arguments
-q / --quiet	Pas de message de bienvenue
-x FICHIER	Exécute des commandes GDB depuis un fichier
-ex COMMANDE	Exécute une commande GDB
-batch	Mode batch (avec -x ou -ex)
-tui	Interface texte avec fenêtres
-symbols FICHIER	Charge les symboles depuis ce fichier
--pid PID	Attache à un PID

COMMANDES GDB ESSENTIELLES :

run [ARGS]	Lance le programme (r)
quit	Quitte GDB (q)
break EMPLACEMENT	Point d'arrêt (b) : fonction, fichier:ligne, *adresse
tbreak EMPLACEMENT	Point d'arrêt temporaire (s'efface après 1 hit)
watch VARIABLE	Watchpoint : pause quand la variable change
delete [N]	Supprime le breakpoint N (ou tous)
enable / disable N	Active/désactive un breakpoint
continue	Continue l'exécution (c)
next	Instruction suivante sans entrer dans les fonctions (n)
step	Instruction suivante, entre dans les fonctions (s)
nexti / stepi	Instruction machine suivante (ni / si)
finish	Exécute jusqu'à la fin de la fonction courante
until [N]	Exécute jusqu'à la ligne N
return [VALEUR]	Retourne de la fonction courante
backtrace	Pile d'appels (bt)
bt full	Pile avec variables locales
frame N	Sélectionne le frame N
up / down	Monte/descend dans la pile
info locals	Variables locales du frame courant
info args	Arguments du frame courant
info registers	Registres CPU
info threads	Liste des threads
thread N	Change de thread
print EXPR	Affiche la valeur d'une expression (p)
display EXPR	Affiche à chaque pause
set variable X = Y	Modifie une variable
x/NF ADRESSE	Examine la mémoire (N éléments, format F, adresse)
list [EMPLACEMENT]	Affiche le code source (l)
disassemble [EMPLACEMENT]	Désassemble (disas)
info breakpoints	Liste les breakpoints
info sharedlibrary	Bibliothèques chargées
call FONCTION(ARGS)	Appelle une fonction du programme
signal NOM	Envoie un signal au processus
signal 0	Vérifie si le processus existe
generate-core-file	Génère un core dump
checkpoint / restart N	Snapshots d'état (si supporté)

EXEMPLES :

\$ gdb mon_programme	# Lance le débogueur
\$ gdb mon_programme core.1234	# Analyse un core dump
\$ gdb -p 1234	# Attache à un processus
\$ gdb -batch -ex "bt" mon_programme core.1234	# Backtrace non-interactive
# Session typique post-crash :	
(gdb) bt	# Voir la pile au moment du crash
(gdb) frame 3	# Aller au frame 3
(gdb) info locals	# Variables locales
(gdb) list	# Code source autour du crash
# Session typique de débogage :	
(gdb) break main	# Breakpoint sur main
(gdb) run	# Lance le programme
(gdb) next	# Avance ligne par ligne
(gdb) print ma_variable	# Voir la valeur

```
[USER] coredumpctl
```

DESCRIPTION : Gère les core dumps collectés par systemd-coredump. Permet de les lister, inspecter et déboguer avec GDB.

POURQUOI : systemd-coredump capture automatiquement les crashes des programmes. coredumpctl permet de retrouver, inspecter et analyser ces crashes sans chercher manuellement les fichiers core.

CONTEXTE : Analyse post-crash, maintenance, débogage de production.

COMMANDES :

- list [PATTERN] Liste les core dumps (filtre par exe, PID, uid, signal)
- info [PATTERN] Détails d'un core dump (signaux, backtrace, métadonnées)
- debug [PATTERN] Ouvre dans GDB
- dump [PATTERN] Exporte le core dump vers un fichier ou stdout
- gdb [PATTERN] Alias de debug

OPTIONS :

- r / --reverse Tri inverse (plus récent en premier)
- l Seulement le plus récent
- n N N entrées
- q / --quiet Minimal
- S / --since= Depuis cette date
- U / --until= Jusqu'à cette date
- o FICHIER Fichier de sortie pour dump

EXEMPLES :

```
$ coredumpctl list # Tous les crashes récents
$ coredumpctl list --reverse -n 10 # 10 plus récents
$ coredumpctl list python3 # Crashes de python3
$ coredumpctl info # Détails du dernier crash
$ coredumpctl info /usr/bin/gnome-shell # Crashes de GNOME Shell
$ coredumpctl debug # Ouvre le dernier crash dans GDB
$ coredumpctl debug 1234 # Ouvre le crash du PID 1234 dans GDB
$ coredumpctl dump > core.bin # Exporte le dernier core dump
```

I. COLLECTE D'INFORMATIONS DE SUPPORT

[SUDO] sosreport

DESCRIPTION : Collecte un rapport exhaustif de l'état du système (logs, config, hardware, services, performances) dans une archive compressée.

POURQUOI : Fournir toutes les informations nécessaires au support technique (Red Hat, SUSE, etc.) ou à la communauté pour diagnostiquer un problème. Évite les aller-retours "envoyez ce fichier, maintenant celui-là..."

CONTEXTE : Support Red Hat Enterprise Linux / Fedora / CentOS / Rocky Linux.

OPTIONS :

- l / --list-plugins Liste tous les plugins disponibles
- n / --skip-plugins LISTE Ignore ces plugins
- e / --enable-plugins LISTE Active ces plugins supplémentaires
- o / --only-plugins LISTE N'exécute que ces plugins
- batch Mode non-interactif (pas de questions)
- no-report Pas de rapport HTML
- q / --quiet Mode silencieux
- case-id ID Numéro de ticket support (pour le nommage)
- label ÉTIQUETTE Étiquette personnalisée
- z TYPE Compression (auto, gzip, bzip2, xz, lzma, zip)
- C CHEMIN Répertoire de sortie
- config-file FICHIER Fichier de configuration alternatif
- t / --threads N Nombre de threads
- timeout N Timeout global en secondes
- plugin-timeout N Timeout par plugin
- v / --verbose Verbeux
- all-logs Collecte tous les logs (pas de limite de taille)
- since YYYY-MM-DD Logs depuis cette date
- allow-system-changes Permet les modifications système (certains plugins)

EXEMPLES :

```
$ sosreport # Rapport complet (interactif)
$ sosreport --batch # Non-interactif
$ sosreport --batch --case-id 123456 # Avec numéro de ticket
$ sosreport -o networking,hardware # Seulement ces plugins
$ sosreport -n rpm,yum # Skip les plugins RPM/YUM (lents)
$ sosreport --all-logs --batch # Tous les logs, sans questions
# Le rapport est créé dans /var/tmp/sosreport-*.tar.xz
```